

Introduction to Java

- Java is a free-to-use and highly dynamic high-level programming language for coding programs such as web applications, mobile applications, enterprise software, big data applications, and server-side technologies.
- It was created by Sun Microsystems in 1995 as a multi-platform, object-oriented, and network-centric language that can be used as a platform in itself.
- **Common uses of Java:** Game development, cloud computing, big data, artificial intelligence (AI), and Internet of Things.

Java Runtime Environment (JRE)

- It is the software that Java programs require to run correctly. It provides the class libraries and other resources that any Java program needs.

Advantages of Using Java

- **Active Community Support:** Given its popularity and longevity, Java has many active users and a community that can support developers.
- **High-Quality Learning Resources:** Learning resources such as detailed documentation, comprehensive books, and courses are available to new programmers for additional support.
- **Inbuilt Functions and Libraries:** Developers do not need to write every new function from scratch, as Java provides a rich ecosystem of built-in functions and libraries to develop various applications.
- **Security:** The Java security levels and restrictions are highly configurable. An untrusted Java code can be downloaded and run in a secure environment in which it cannot do any damage. The untrusted code also cannot damage the host system with a virus, nor can it read or write files from the hard drive.
- **Platform Independent:** Java code can be written once and run on any underlying platform like Windows, iOS, Android, and Linux without rewriting. This feature is called the “Write Once, Run Anywhere” (WORA) Philosophy which became a highly sought feature by developers.

Key Components to Understand Java

- To use Java, a developer must understand two (2) things: **Java Virtual Machine** and **Java Language and APIs**.

Java Virtual Machine (JVM)

- It is a virtual machine that runs Java applications as a run-time engine.
- Natural programming languages fall into two (2) broad categories: compilers and interpreters.
 - **Compilers:** The program is written in natural English-like syntax with compilers, and the language then translates the entire code into machine language. It converts the source code into a binary program of bytecode.

- **Interpreters:** This checks the bytecode and communicates with the operating system, executing the bytecode instructions line by line.
- Java was the first programming language to combine both using a Java Virtual Machine (JVM). A JVM is an environment that translates Java bytecode, an intermediate-level language, into machine language and executes it.
- The programming statements or source code are stored on a disk with a **.java** file extension.
- The Java compiler converts the source code into a binary program of bytecode. The Java interpreter will then check the bytecode and communicate with the operating system, executing the bytecode instructions line by line within the Java Virtual Machine, named *java.exe*.

Java APIs

- APIs (Application Programming Interface) are important software components bundled with the Java Platform.
- Java APIs are pre-written Java programs that plug and play existing functionality into a code, such as getting the time and date, performing mathematical operations, or manipulating text.
- The Java language has only about 50 keywords, but the Java API has several thousand classes, with thousands of methods that can be used in any program.

Java Skeleton

- The best way to understand a programming language is by dissecting and examining its sample program.

```
class Skeleton {
    public static void main (String[] args){
        System.out.println("First Java Application"); //for display
    }
}
```

Class & Class Name

- **Class:** A class is the basic unit of a Java program. It also defines a group of objects that includes data members, which are places to store data, and methods which are places to put the program logic. Think of a class as a blueprint for a house and the house itself as the object.
- **Class Name:** A class name is a type of an identifier.

Identifiers

- **Identifier:** An identifier is the name given to a program component, such as a class, object, or variable.

Requirements for Naming an Identifier:

- It must begin with a letter of the English alphabet, an underscore (`_`), or a dollar sign (`$`), but not with a digit.
- It can have any combination of letters, digits, underscores, or dollar signs, but not with white spaces.
- It is case-sensitive such as the identifier `NUMBER` is not the same as the identifier `number` or `Number`.
- It cannot be one of the following values: `true`, `false`, or `null`. These are not keywords; they are primitive values but reserved and cannot be used.
- It cannot be a reserved word.

Reserved Words

- Reserved words or keywords are special words significant to the Java compiler.
- Aside from class names, these cannot be used as variables, a name used to refer to data stored in the memory.

<code>abstract</code>	<code>continue</code>	<code>for</code>	<code>new</code>	<code>switch</code>
<code>assert</code>	<code>default</code>	<code>goto</code>	<code>package</code>	<code>synchronized</code>
<code>boolean</code>	<code>do</code>	<code>if</code>	<code>private</code>	<code>this</code>
<code>break</code>	<code>double</code>	<code>implements</code>	<code>protected</code>	<code>throw</code>
<code>byte</code>	<code>else</code>	<code>import</code>	<code>public</code>	<code>throws</code>
<code>case</code>	<code>enum</code>	<code>instanceof</code>	<code>return</code>	<code>transient</code>
<code>catch</code>	<code>extends</code>	<code>int</code>	<code>short</code>	<code>try</code>
<code>char</code>	<code>final</code>	<code>interface</code>	<code>static</code>	<code>void</code>
<code>class</code>	<code>finally</code>	<code>long</code>	<code>strictfp</code>	<code>volatile</code>
<code>const</code>	<code>float</code>	<code>native</code>	<code>super</code>	<code>while</code>

Braces and Code Blocks

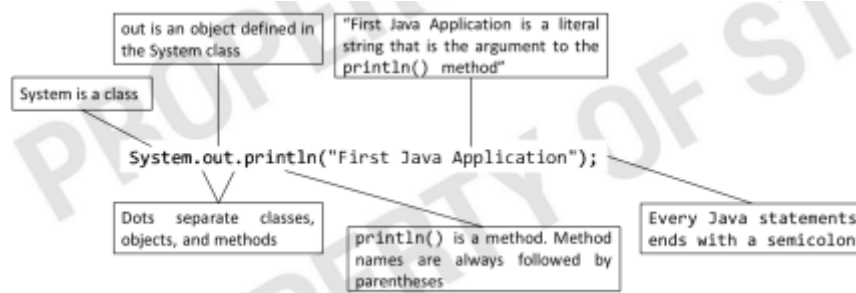
- Braces are used to identify blocks of code and data.
- It requires matching opening (`{`) and closing braces (`}`).
- The whole definition of a class should be between the first opening brace and the last closing brace.

The Main Method Syntax (`public static void main(String[] args)`)

- `public`: A type of access or visibility modifier. This keyword is used to define a `main()` method so that the interpreter can execute the program.
- `static`: Tells that this is a class method. The main method is always declared static so that it can be executed without creating an object of the class.
- `void`: Means that the main is a non-value-returning method, which means the main is not expected to return any value.
- `main`: The name of the method. When executing a Java program, the execution must always begin with the main method. All Java applications must include a class containing one (1) main method.

Method Parameters and Data Structures

- **Parameters:** Listed inside the parentheses after the method name along with their types to allow the method to receive values.
- **args:** A parameter in the main method, which is an array of type **String**.
- **Square Brackets ([]):** Indicate that **args** is an array.
- **Array:** A collection of similar data types.
- **String:** A sequence of characters containing letters, numbers, and symbols.



Literal String, Arguments, and Statements

- **Literal String:** A series of characters that appear exactly as written and is always enclosed by double quotation marks ("").
- **Arguments:** Information passed to a method so it can perform its task.
- **Java Statement:** An executable instruction that commands the compiler what to do.

Comments

- **Comments:** Provide additional information to make programs easier for programmers and other programmers to understand. Comments will not be compiled and executed when the program is run.
- **Single-line Comment:** Starts with // symbols and continues to the end of the line.
- **Multi-line Comment:** Placed between /* and */ symbols if the comments run over multiple lines.

Saving, Compiling, and Executing a Program

- **Java Development Kit (JDK):** Includes complete JRE tools for developing, debugging, and monitoring Java applications. JDK must be installed on the computer to write, compile, and execute Java programs.